

SEARCHING & BOUNDS CHALLENGE SET

Problem Design Exercise — Linear Search, Binary Search, and Boundary Analysis

Difficulty Escalation: +10% Incremental Growth per Problem

Problem 1: Library Audit

DIFFICULTY: BASE (100%)

A university archives department maintains a strictly sorted array of unique, 4-digit historic manuscript IDs. To analyze the efficiency of modern searching algorithms, the archivist requests a benchmark program that compares basic linear and binary searching routines.

TASKS

- Read an integer N representing the number of manuscripts, followed by a sorted array of unique manuscript IDs.
- Read a specific target manuscript ID to find.
- Implement a user-defined function for **Linear Search** that returns the index of the element and counts the total number of operations/comparisons performed.
- Implement a user-defined function for **Binary Search** that returns the index of the element and counts the total number of operations/comparisons performed.
- Display both search results along with their respective comparison counts.

Sample Input	Sample Output
5 1012 2045 3089 4412 5901 4412	Linear Search Index: 3 (Comparisons: 4) Binary Search Index: 3 (Comparisons: 2)

Problem 2: Warehouse Restocking & Insertion

DIFFICULTY: +10%

A localized retail inventory system holds a sorted list of unique item codes currently available in the warehouse. When a new product arrives, the manager needs to know if the product code already exists. If it does not exist, the system must identify the exact index where this new item should be inserted to keep the warehouse list perfectly sorted.

TASKS

1. Read the size of the catalog N and the sorted array of unique item codes.
2. Read a query code representing the incoming product.
3. Implement a custom **Binary Search** to verify if the product code exists.
4. Implement a user-defined **Lower Bound** function that finds the first position containing a value greater than or equal to ($\geq$) the query code. This acts as the correct sorted insertion point if the item is missing.
5. Display whether the item exists, and output its current or intended insertion position.

Sample Input	Sample Output
6 100 150 200 250 300 350 225	Product Status: Not Found Sorted Insertion Index: 3

Problem 3: Logistics Logistics & Batch Counters

DIFFICULTY: +20%

A shipping terminal manages standardized cargo container batches. Containers arriving from different ships are indexed by their exact weight (in metric tons). Since multiple containers can have identical weights, the array contains duplicate elements but remains sorted. The terminal dispatcher wants to look up a target weight and determine the exact number of containers currently match that weight.

TASKS

1. Read the total container count N followed by the sorted array of container weights (with duplicates).
2. Read the target weight query.
3. Implement a user-defined **Lower Bound** function to locate the first occurrence of a weight $\geq$ query.
4. Implement a user-defined **Upper Bound** function to locate the first occurrence of a weight $>$ query.
5. Using the indices returned by your boundary functions, calculate and display the precise count of containers matching the target weight without iterating linearly.

Sample Input	Sample Output
8 12 15 20 20 20 25 30 30 20	First Occurrence Index: 2 Strictly Greater Index: 5 Total Matching Containers: 3

Problem 4: Flight Schedule Windowing

DIFFICULTY: +30%

An airport operations terminal logs scheduled arrivals as timestamps represented by integers (minutes elapsed since midnight). The tracking logs are sorted in ascending order. Air traffic controllers often need to run safety queries to determine how many flights are scheduled to arrive within a closed time window $[A, B]$ (inclusive of both boundaries).

TASKS

1. Read the total flight logs count N and the sorted array of arrival timestamps.
2. Read two query integers representing the lower bound time A and upper bound time B .
3. Write a **Lower Bound** function optimized to find the first index where timestamp $\geq A$.
4. Write an **Upper Bound** function optimized to find the first index where timestamp $> B$.
5. Formulate an expression using both boundaries to return the exact number of flights landing inside the safe operational window.

Sample Input	Sample Output
7 180 240 240 300 360 420 500 200 380	Flights In Range: 4

Problem 5: Automated Valet Proximity Finder

DIFFICULTY: +40%

An automated parking structure tracks active smart-valet bays distributed along a linear tracks system. The system maintains a sorted array of occupied bay positions (measured in meters from the entrance hub). When a new vehicle requests parking, the system attempts to find the single existing occupied bay that is **closest in distance** to the vehicle's preferred arrival coordinate X to run a sync-calibration routine.

TASKS

1. Read the count of occupied parking positions N and the sorted coordinate array.
2. Read the driver's preferred arrival coordinate query X .
3. Use custom boundary functions to identify the closest element *before or equal* to X and the closest element *strictly after* X .
4. Evaluate and compare the absolute structural distances to select the index of the absolute closest parking coordinate. If there is a tie in distance, favor the smaller coordinate index.

Sample Input	Sample Output
5 15 28 40 62 85 35	Closest Occupied Bay Index: 2 Bay Coordinate Value: 40 Absolute Distance: 5